CI Pipeline Demo (Flask + GitHub Actions)

A small Flask service with a four-stage GitHub Actions pipeline: lint, test, build and package.

The app

Endpoint	What it does
`GET /`	Returns a greeting
`GET /health`	Returns `{"status": "ok"}`
`GET /add?a=2&b=3`	Returns the sum, or a 400 error if the input is not numbers

Run it locally

You need Python 3.12.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

flask --app ciapp.app run        # run the app on http://127.0.0.1:5000
flake8 .                         # lint (settings are in .flake8)
python -m pytest -v              # run the 5 unit tests
python -m build --wheel          # build: creates dist/ciapp-1.0.0-py3-none-any.w
```

To package it in Docker (the wheel must be in `dist/` first):

```
docker build -t ci-demo .
docker run -p 5000:5000 ci-demo
```

How the pipeline works (`.github/workflows/ci.yml`)

- **When it runs:** on every pull request to `main` , every push to `main` , and tags starting with `v` .
- **lint** runs flake8. **test** runs pytest. **build** builds the wheel and uploads `dist/` as an

artifact. **package** downloads that artifact and builds the Docker image.

- Each job has `needs:` on the one before it, so a failure stops the rest.
- `package` only runs on pushes, never on pull requests.
- pip's download cache is stored between runs, and a newer commit on the same branch cancels the older run.
- The workflow only has `contents: read` permission, and no secrets are in the repository.

Evidence

- **Green run, all four jobs passing:** https://github.com/Oreoluwa26/ci-demo/actions/runs/41188476228
- **Failing PR 1, lint error:** https://github.com/Oreoluwa26/ci-demo/pull/1
- **Failing PR 2, failing test:** https://github.com/Oreoluwa26/ci-demo/pull/2
- **Failing PR 3, broken build:** https://github.com/Oreoluwa26/ci-demo/pull/3
- **Cold run (no cache):** https://github.com/Oreoluwa26/ci-demo/actions/runs/41188476228/attempts/1 took **48 seconds**
- **Warm run (same commit, cache hit):** https://github.com/Oreoluwa26/ci-demo/actions/runs/41188476228/attempts/2 took **1 minute 6 seconds**

Pipeline design

1. Stage order

I put the quickest and cheapest check first. Lint takes a few seconds, tests take a bit longer, and building and packaging are the slowest, so a typo never costs a full build. I ran lint and test in sequence, not in parallel. In parallel, the whole pipeline would finish sooner, because it would take as long as the slower of the two jobs. But if lint fails, the test job would still have used runner minutes for nothing. In sequence, a failure stops the pipeline early and costs less. In this project each job takes only about 8 seconds, so waiting a few extra seconds is a small price for lower cost and a clearer result.

2. Caching

I cache pip's download cache, which is the packages pip downloads. The cache key is made by `actions/setup-python`. In my run it was `setup-python-Linux-x64-26.04-Ubuntu-python-3.12.15-pip-3044b0ae...` : the operating system, the Python version, `pip`, and a hash of `requirements.txt`. The cache is about 3 MB. When the lockfile changes, the hash changes, so no cache matches. The job installs everything fresh and saves a new cache. If the key was a fixed string like `deps`, the cache would never be refreshed. After I changed a dependency, the pipeline would keep restoring the old packages, so it could pass or fail on

versions that don't match the lockfile.

3. Fail-fast

- **PR 1 (lint error):** `lint` ran and failed. `test`, `build` and `package` were skipped.
- **PR 2 (failing test):** `lint` passed, and `test` ran and failed. `build` and `package` were skipped.
- **PR 3 (broken build):** `lint` and `test` passed, and `build` ran and failed. `package` was skipped.

This comes from the `needs:` lines. `test` has `needs: lint`, `build` has `needs: test`, and `package` has `needs: build`. When a job fails, every job that needs it is skipped. On pull requests `package` is also skipped by `if: github.event_name == 'push'`.

4. Speed

The cold run (no cache) took 48 seconds in total (lint 8s, test 8s, build 9s, package 9s). The warm run on the same commit took 1 minute 6 seconds (lint 11s, test 8s, build 9s, package 18s). The log of the warm run says "Cache restored successfully", so the cache worked, but the warm run was not faster. My project only has about 3 MB of packages, so restoring them from the cache saves very little, and the rest of the difference came from things the cache doesn't control: GitHub's runners vary in speed and start-up time, and the package job (which builds the Docker image) took longer on that run. In a bigger project with many large packages, the cache would save much more time.

Cancelling older runs (`concurrency` with `cancel-in-progress: true`) helps when someone pushes three commits in two minutes. Only the last commit runs to the end, and the first two are cancelled. That saves runner minutes on code that is already out of date, and the developer gets the result that matters sooner.

5. Safety

If the package job pushed the image, it would read the registry password from GitHub Actions encrypted secrets (Settings, then Secrets and variables, then Actions), as `${{ secrets.REGISTRY_PASSWORD }}`. It would never be written in the repository. Pull requests from forks should not get secrets because anyone can fork a public repo and open a pull request that changes the workflow to print the secret. GitHub does not pass secrets to workflows from fork pull requests for this reason, and my package job runs only on pushes.

6. Improve it

- **Install the pinned versions in the Docker image.** My Dockerfile installs the wheel, which pulls the latest Flask, not the version in `requirements.txt`. The image could behave differently from what the tests used. Installing from the pinned file fixes this.

- **Push the image to a registry (GitHub Container Registry) when a `v` tag is pushed.** At the moment the image is built but never published, so nothing can be deployed from it.

Assumptions

- I chose Python 3.12 and Flask.
- The image is built but not pushed to a registry, so no registry password is needed.
- `requirements.txt` holds the pinned packages for the app and the dev tools.
- The three failing pull requests are left open on purpose, as evidence.